UNIVERSIDADE FEDERAL DO PARÁ
CAMPUS UNIVERSITÁRIO DE TUCURUÍ
FACULDADE DE ENGENHARIA DE COMPUTAÇÃO

WENDEL MELO LEÃO
WELLINGTON NOGUEIRA LOPES

RELATÓRIO DO PROJETO DE TEORIA DA COMPUTAÇÃO

TUCURUÍ

2026

1 INTRODUÇÃO

Autômatos finitos são modelos matemáticos utilizados para reconhecer linguagens regulares. Eles possuem aplicação direta em compiladores, analisadores léxicos e ferramentas de busca textual, como o comando grep em sistemas Unix. Segundo Hopcroft, Motwani e Ullman (2002), os autômatos constituem a base formal para o estudo das linguagens regulares.

2 DEFINIÇÕES FORMAIS E DETERMINISMO

Um Autômato Finito Determinístico (AFD) é definido como a quíntupla $M = (Q, \Sigma, \delta, q0, F)$, onde $\delta: Q \times \Sigma \rightarrow Q$. Já o Autômato Finito Não Determinístico (AFND) possui a função de transição $\delta: Q \times \Sigma$

$\rightarrow P(Q)$, permitindo múltiplos estados destino (Sipser, 2011). No AFD existe apenas um caminho possível de execução para cada símbolo lido. Já no AFND, vários caminhos podem ser explorados simultaneamente.

3 CONVERSÃO AFND PARA AFD

A conversão de AFND para AFD utiliza o algoritmo de construção de subconjuntos. Nesse processo, cada estado do AFD representa um conjunto de estados do AFND. O algoritmo inicia com o estado inicial do AFND e constrói iterativamente novos estados representando combinações de estados possíveis. Essa abordagem garante que o autômato resultante seja determinístico e reconheça a mesma linguagem.

3.1 Pseudocódigo simplificado:

```
inicial ← {q0}
fila ← [inicial]

enquanto fila não estiver vazia:
                                                    T ← remover da fila
    para cada símbolo a em Σ:
         U ← união de δ(q,a) para todo q ∈ T
                          se U não visitado:
            adicionar U na fila
        δ(T,a) ← U
```

4 PRINCIPAIS FUNÇÕES AFND -> AFD

Função *fecho_epsilon ()*

Calcula o conjunto de estados alcançáveis a partir de transições vazias, fundamental para processar AFN com movimentos vazios.

Função *mover()*

Determina o conjunto de estados alcançáveis a partir de um conjunto de estados atual dado um símbolo específico do alfabeto.

Função *afn_para_afd* ()

Implementa o algoritmo de construção de subconjuntos. Cria estados compostos no AFD para representar as múltiplas possibilidades do AFND.

5 MINIMIZAÇÃO DE AFD

Após a conversão para AFD, aplica-se o processo de minimização. A minimização remove estados redundantes mantendo a mesma linguagem reconhecida. Estados são considerados equivalentes quando apresentam comportamento idêntico para todos os símbolos do alfabeto. O algoritmo de Moore utiliza refinamento sucessivo de partições para separar estados distinguíveis.

5.1 Etapas principais do algoritmo de Minimização:

```
1. Remover estados inalcançáveis
2. Garantir função de transição total
3. Criar partições iniciais (finais e não finais)
4. Refinar partições
```

6 PRINCIPAIS FUNÇÕES MINIZAÇÃO AFD

Função *remover_inalcancaveis* ()

Realiza uma busca em largura/profundidade apara eliminar estados que nunca podem ser atingidos.

Função *tornar_total_com_sumidouro*()

Garante que o AFD tenha transições para todos os símbolos, criando um "estado sumidouro" (trap state) se necessário

Função *minimizar_afd ()*

Implementa o particionamento de estados (Algoritmo de Moore/Hopcroft). Divide os estados em grupos de equivalência baseados em seu comportamento de aceitação e transição.

7 REPRESENTAÇÃO COMPUTACIONAL

Na implementação em Python, os estados são armazenados utilizando estruturas do tipo set. A função de transição δ é implementada como um dicionário cuja chave é composta pelo par (estado, símbolo). Essa estrutura permite acesso direto em tempo constante e facilita operações de união de conjuntos necessárias durante a construção do AFD.

7.1 Exemplo conceitual de tabela de transição:

```
transicoes = {
(q0,'a'): {q1},
(q1,'a'): {q2},
(q2,'b'): {q3}
}
```

8 EXEMPLO DE EXECUÇÃO DO AUTÔMATO

Durante a execução do programa, o autômato processa a palavra símbolo por símbolo. A cada etapa é aplicada a função de transição δ para determinar o próximo estado.

```
Entrada: aabbab

PASSO 0: P0 --a--> P1
PASSO 1: P1 --a--> P4
PASSO 2: P4 --b--> P2
PASSO 3: P2 --b--> P2
PASSO 4: P2 --a--> P3
PASSO 5: P3 --b--> P6

Estado final: P6
Resultado: ACEITA
```

CONCLUSÃO

A conversão de AFND para AFD demonstra que o não determinismo é um recurso de modelagem. A minimização reduz a complexidade estrutural do autômato sem alterar a linguagem reconhecida. Esses conceitos são amplamente aplicados em compiladores, motores de busca textual e processadores de linguagem.

REFERÊNCIAS

AHO, A.; LAM, M.; SETHI, R.; ULLMAN, J. Compiladores: princípios, técnicas e ferramentas. 2. ed. São Paulo: Pearson, 2006. HOPCROFT, J.; MOTWANI, R.; ULLMAN, J. Introdução à teoria de autômatos, linguagens e computação. 2. ed. São Paulo: Pearson, 2002. SIPSER, M. Introdução à teoria da computação. 2. ed. São Paulo: Cengage Learning, 2011.